

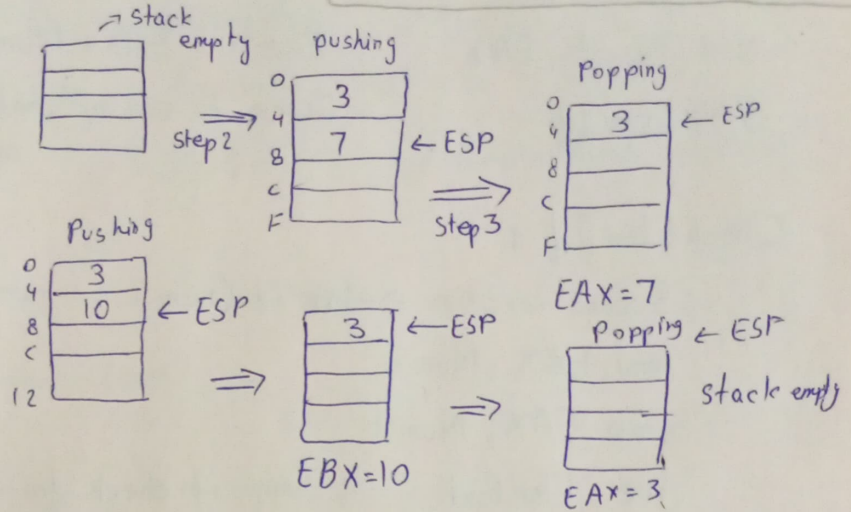
COAL-Assignment 2

Name :- Sara Munir Alam

Roll#:- 24K-0573

Class :- BCS-3E

Q1) `mov eax, 3`
`mov ebx, 7`
`push eax`
`push ebx`
`pop eax`
`push 10`
`pop ebx`



∴ After these instruction Execution

EAX = 3

Stack = Empty / Garbage values

EBX = 10

Q2) • data

Num1 DWORD ?

Num2 DWORD ?

Num3 DWORD ?

Num4 DWORD ?

Result DWORD ?

• code

main PROC

; Assume Num1, Num2, Num3, Num4 are already initialised with values.

; First Condition check : If (Num1 > Num2 && Num3 != Num4)

`mov EAX, Num1`

`cmp EAX, Num2`

`jle CheckElseIf` ; Jump if check fails (num1 <= num2)

; Second Part of first condition : Num3 != Num4

`mov EBX, Num3`

`cmp EBX, Num4`

`JE CheckElseIf` ; Jump if check fails (num3 == num4)

; First condition is true

mov EAX, Num1

add EAX, Num3

mov Result, EAX ; Result = EAX + Num3 = Num1 + Num3

JMP EndIf ; Jump to end of condition check

CheckElseIf :

; Second condition $\Rightarrow$ else if (num2 == num4)

mov EAX, Num2

cmp EAX, Num4

jne ElsePart ; Jump if check fails (Num2 $\neq$ Num4)

; Second condition check successful

mov EAX, Num2

sub EAX, Num1

mov Result, EAX ; Result = EAX - Num1 = Num2 - Num1

JMP EndIf ; Jump to end of condition check

ElsePart :

mov Result, 0 ; Result = 0

EndIf :

; continue with rest of program (assuming result is displayed)

mov EAX, Result

Call WriteInt

exit

main ENDP

END main

Q3)

• data

Sum SDWORD 0 ; Initialising Sum = 0

i DWORD 0 ; Initialising i = 0

Count DWORD 5 ; Assuming Count = 5 Hardcode

Array SDWORD 7, -3, 15, 0, -8 ; Example Array Hardcode

• code

main PROC

; Starting While Loop

WhileLoop:

mov ECX, i

cmp ECX, Count ; compare ECX, count = i and count

jge EndWhile ; exit loop if ECX > Count, i > count

; Calculate array[i] address and load value

mov esi, OFFSET array ; ESI points to start of array

mov EAX, i

imul eax, 4 ; Multiply index by 4 cause SDWORD

add esi, eax ; ESI now points to array[i]

mov EBX, [ESI] ; EBX = Array[i]

; Check if Array[i] > 0

cmp EBX, 0

jle ElsePart ; Jump if condition fails (Array[i] < 0)

; If condition is true then Sum = Sum + Array[i]

mov EAX, Sum

add EAX, EBX ; EAX = Sum + Array[i]

mov Sum, EAX ; Sum = Sum + Array[i]

JMP Increment_i ; jump to increase i counter

Else Part :

```
; else { Sum = Sum - 1)
mov EAX, Sum
dec EAX
mov Sum, EAX
```

Increment i :

```
move EAX, i ; EAX = i
inc EAX ; EAX = EAX + 1
mov i, EAX
JMP WhileLoop ; Jump back to start of loop
```

End While ; ; assuming after loop ends result is displayed
mov EAX, Sum ; Displaying result
call WriteInt

exit

main ENDP

END main

Q4) Initial State

AX = 0000H

ECX = 0AH (10 in decimal)

Iteration 1

AX = 0000H - 1 = FFFFH

ECX = 09H

Iteration 2

AX = FFFFH - 1 = FFFE H

ECX = 08H

Iteration 3

AX = FFFE H - 1 = FF FDH

ECX = 07H

Iteration 4

AX = FFFCH

ECX = 06H

Iteration 5

AX = FFFBH

ECX = 05H

Iteration 6

AX = FFFAH

ECX = 04H

Iteration 7

AX = FFF9H

ECX = 03H

Iteration 8

AX = FFF8H

ECX = 02H

Iteration 9

AX = FFF7H

ECX = 01H

Iteration 10

AX = FFF6 H

ECX = 00 H loop terminate

$\therefore$ AX = FFF6 H

Q5) AL = 0D4 H

SHL AL, 3

sol

Binary form :-

	1	1	0	1	0	1	0	0	
CF	/	/	/	/	/	/	/	/	insert initial
[1]	1	0	1	0	1	0	0	0	1 st shift
CF	/	/	/	/	/	/	/	/	
[1]	0	1	0	1	0	0	0	0	2 nd shift
CF	/	/	/	/	/	/	/	/	
[0]	1	0	1	0	0	0	0	0	3 rd shift

$\therefore$ AL = 0A0H after SHL AL, 3

MOV AL, 0D4 H

SAL AL, 3

Note :- SAL and SHL operation is nearly same so result will be same

sol

Binary form :-

	1	1	0	1	0	1	0	0	
CF	/	/	/	/	/	/	/	/	insert initial
[1]	1	0	1	0	1	0	0	0	1 st shift
CF	/	/	/	/	/	/	/	/	
[1]	0	1	0	1	0	0	0	0	2 nd shift
CF	/	/	/	/	/	/	/	/	insert
[0]	1	0	1	0	0	0	0	0	3 rd shift

$\therefore$ AL = 0A0H after SAL AL, 3

STC ; setting carry flag to 1

MOV AL, 0D4

ROL AL, 1

Sol :

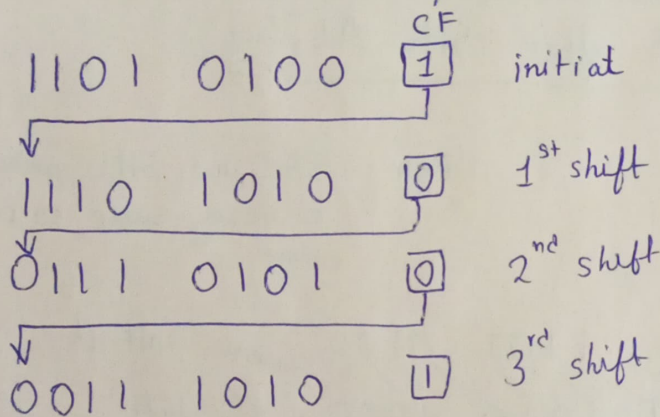
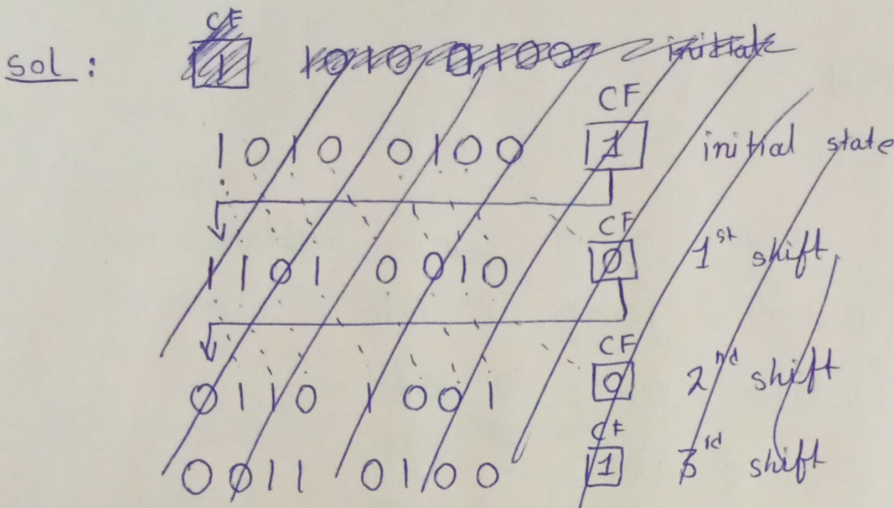
CF		0		4					
[1]	1	1	0	1	0	1	0	0	initial form
[1]	1	0	1	0	1	0	0	1	rotate left once

$\therefore$ AL = 0A9H after ROL AL, 1

STC ; setting CF to 1

Mov AL, 0D4H

RCR AL, 3



∴ AL = 3Ah after RCR AL, 3

Q6) • data

1 op1 SDWORD 22
2 op2 SDWORD 36
3 x SDWORD -10
4 y SDWORD 29
5 z SDWORD 0

• code

7 main PROC

8 While Loop :

9 mov EAX, op1

10 cmp EAX, op2

; Assuming values for variables

11 JG ENDWhile

12 mov EAX, op1

13 cmp EAX, x ; If (op1 >= x) &

14 JL ElsePart ; if first condition fails

15 cmp eax, y ; if (op1 <= y)

16 JG ElsePart ; if second condition fails

17 mov eax, z ; if both condition fail

18 add eax, 10

19 mov z, eax


```

line
20 JMP Decrement
21 ElsePart;
22     mov EAX, Z
23     sub EAX, 10 ; Z = Z - 10
24     mov Z, EAX
25 Decrement;
26     mov eax, op1
27     dec eax ; op1--
28     mov op1, eax
29 JMP WhileLoop ; continue loop
30 ENDWhile;
31     exit
32 main ENDP
33 End Main

```